

Week 03 - Model Context Protocol (MCP)

Overview

This week introduces the **Model Context Protocol (MCP)**, an open-source standard for connecting AI applications to external systems. MCP enables AI applications to access data sources, tools, and workflows, making them more capable and personalized.

What is MCP?

The Model Context Protocol (MCP) is like a **USB-C port for AI applications**. Just as USB-C provides a standardized way to connect electronic devices, MCP provides a standardized way to connect AI applications to external systems.

Key Benefits

- **For Developers:** Reduces development time and complexity when building AI applications
- **For AI Applications:** Provides access to an ecosystem of data sources, tools, and apps
- **For End Users:** Results in more capable AI applications that can access your data and take actions

What MCP Can Enable

- Agents can access your Google Calendar and Notion, acting as a personalized AI assistant
- Enterprise chatbots can connect to multiple databases across an organization
- Custom financial data servers for real-time market analysis

Prerequisites

Python Installation

Before starting, ensure you have Python 3.12 or higher installed on your system.

Windows Installation

1. Visit python.org
2. Download Python 3.12 for Windows
3. Run the installer and check "Add Python to PATH"
4. Verify installation:

```
python --version
```

macOS Installation

1. Install Python:

```
brew install python@3.12
```

2. Verify installation:

```
python3 --version
```

Linux Installation

1. Update package list:

```
sudo apt update
```

2. Install Python 3.12:

```
sudo apt install python3.12 python3.12-venv python3.12-pip
```

3. Verify installation:

```
python3.12 --version
```

UV Package Manager

UV is a fast Python package installer and resolver. Install it:

macOS/Linux

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Restart your terminal after installation and verify:

```
uv --version
```

Part 1: Building an MCP Server

Step 1: Create Server Project

```
# Create a new directory for our server project
uv init mcp-server
cd mcp-server

# Create virtual environment and activate it
uv venv
source .venv/bin/activate # On Windows: .venv\Scripts\activate

# Install dependencies
uv add "mcp[cli]" httpx # On Windows: uv add mcp[cli] httpx

# Create our server file
touch financial_data_server.py # On Windows: new-item financial_data_server.py
```

Step 2: Understanding MCP Tools

MCP servers can provide three main types of capabilities:

1. **Resources:** File-like data that can be read by clients
2. **Tools:** Functions that can be called by the LLM (with user approval)
3. **Prompts:** Pre-written templates that help users accomplish specific tasks

Our server focuses on **Tools**. The `@mcp.tool()` decorator automatically generates tool definitions from Python functions using type hints and docstrings.

Step 3: Server Implementation

Copy the provided `financial_data_server.py` code into your file. This server includes:

- `get_cryptocurrency_price`: Real-time crypto prices via CoinGecko API
- `get_company_profile`: Company data via Alpha Vantage API
- `get_crypto_market_overview`: Top cryptocurrencies market overview

Step 4: Running the Server

```
# Run the server in a separate terminal window
uv run financial_data_server.py
```

The server will start and listen for MCP client connections via stdio transport.

Part 2.1: Testing with Claude Desktop (Optional)

Since we'll build a custom client, this step is optional, but it can be useful for testing **the server** with a ready-made interface.

Claude Desktop Configuration

1. Install [Claude Desktop](#)
2. Open configuration file:

```
# macOS
code ~/Library/Application\ Support/Claude/claude_desktop_config.json

# Windows
code $env:AppData\Claude\claude_desktop_config.json
```

3. Add server configuration:

```
{
  "mcpServers": {
    "financial_data_server": {
      "command": "uv",
      "args": [
        "--directory",
        "/ABSOLUTE/PATH/TO/YOUR/mcp-server",
        "run",
        "financial_data_server.py"
      ]
    }
  }
}
```

4. Restart Claude Desktop and test with queries like:

- "What's the current price of Bitcoin?"
- "Get the company profile for IBM"
- "Show me the top 10 cryptocurrencies"

Part 2.2: Testing with Cursor (Optional)

The process is almost the same as with Claude Desktop, but in **Cursor** the configuration is done via the UI.

Cursor Configuration

1. Open **Cursor**.
2. Go to: Preferences > Cursor Settings > MCP > + New MCP Server
3. A field will appear where you can add the same JSON configuration used above:

```
{
  "mcpServers": {
    "financial_data_server": {
      "command": "uv",
      "args": [
        "--directory",
        "/ABSOLUTE/PATH/TO/YOUR/mcp-server",
        "run",
        "financial_data_server.py"
      ]
    }
  }
}
```

Once you've configured your MCP server in Cursor, you can immediately start using it through the Cursor agent interface. This is the fastest way to test your server functionality.

Opening Cursor Agent

1. **On macOS:** Press Command + L (⌘L)
2. **On Windows/Linux:** Press Ctrl + L

This opens the Cursor agent chat interface where you can interact with your MCP server.

Testing Your MCP Server

Once the agent is open, you can immediately start asking questions that will use your financial data server:

```
"What's the current price of Bitcoin?"
"Show me the top 10 cryptocurrencies by market cap"
"Get the market overview for cryptocurrencies"
```

How It Works

1. **You ask a question** in the Cursor agent chat
2. **Cursor analyzes** your question and determines which MCP tools to use
3. **Your server executes** the appropriate tool (e.g., `get_cryptocurrency_price`)
4. **Real data is returned** from the APIs (CoinGecko, Alpha Vantage)
5. **Cursor formats** the response in a natural, conversational way

Example Interaction

You: "What's the current price of Ethereum and how has it performed today?"

Cursor Agent: *[Uses your `get_cryptocurrency_price` tool]*

Response: "Ethereum (ETH) is currently trading at \$4,537.78 USD. In the last 24 hours, it has declined by 1.25%, with a market cap of \$547.68 billion and trading volume of \$26.60 billion."

Benefits of This Approach

- **Immediate Testing:** No need to build a custom client first
- **Natural Interaction:** Ask questions in plain text
- **Real-time Data:** Get live financial data from your server
- **Learning Tool:** See exactly how your MCP tools are being used

This method allows you to validate your server implementation before building the custom client, ensuring everything works correctly.

Part 3: Building an MCP Client

Step 1: Create Client Project

```
# Create project directory (separate from server)
uv init mcp-client
cd mcp-client

# Create virtual environment
uv venv

# Activate virtual environment
# On Windows:
.venv\Scripts\activate
# On Unix or macOS:
source .venv/bin/activate

# Install required packages
uv add mcp python-dotenv langchain langchain-openai langchain-mcp-adapters langgraph

# Remove boilerplate files
# On Windows:
del main.py
# On Unix or macOS:
rm main.py

# Create our main file
touch client.py
```

Step 2: Environment Configuration

Create a `.env` file in the client directory:

```
touch .env
```

Add your OpenAI API key to the `.env` file:

```
OPENAI_API_KEY=sk-proj-your-openai-api-key-here
```

Step 3: Client Implementation

Copy the provided `client.py` code into your file. This client:

- Connects to MCP servers via stdio transport
- Uses LangChain and LangGraph for AI agent functionality
- Provides an interactive chat interface
- Automatically loads and uses MCP tools

Step 4: Running the Client

```
# Run the client, connecting to your server
# You need to provide the absolute path to the server Python file
uv run client.py <ABSOLUTE_PATH_TO_SERVER_FILE>
```

Finding the absolute path:

- **macOS/Linux:** Use `pwd` to get the current directory path, then modify it to point to the server:

```
pwd # Shows current directory (e.g., /Users/username/project/week_03/mcp-client)
# Since we are in mcp-client, replace "mcp-client" with "mcp-server" and add the filename
# Final path: /Users/username/project/week_03/mcp-server/financial_data_server.py
```

- **Windows:** Use `cd` to get the current directory path, then modify it to point to the server:

```
cd # Shows current directory (e.g., C:\Users\username\project\week_03\mcp-client)
# Since we are in mcp-client, replace "mcp-client" with "mcp-server" and add the filename
# Final path: C:\Users\username\project\week_03\mcp-server\financial_data_server.py
```

Example with absolute path:

```
uv run client.py /Users/yourusername/path/to/week_03/mcp-server/financial_data_server.py
```

Usage Examples

Once both server and client are running, you can interact with the financial data tools:

```
Query: What's the current price of Bitcoin?
Query: Show me the top 10 cryptocurrencies by market cap
Query: quit
```

Project Structure

```
week_03/
├── README.md
├── mcp-server/
│   ├── financial_data_server.py
│   ├── pyproject.toml
│   └── .venv/
└── mcp-client/
    ├── client.py
    ├── .env
    ├── pyproject.toml
    └── .venv/
```

Key Concepts

MCP Architecture

- **Servers:** Expose tools, resources, and prompts
- **Clients:** Connect to servers and use their capabilities
- **Transport:** Communication layer (stdio, HTTP, etc.)
- **Protocol:** JSON-RPC based communication

Tool Definition

```
@mcp.tool()
async def get_crypto_currency_price(symbol: str, currency: str = "usd") -> str:
    """
    Get current cryptocurrency price and market data.

    Args:
        symbol: Cryptocurrency symbol (e.g., 'bitcoin', 'ethereum')
        currency: Target currency for price display (default: 'usd')

    Returns:
        Formatted string with current price and market data
    """
    # Implementation here
```

The `@mcp.tool()` decorator automatically:

- Generates tool metadata from function signature
- Uses docstring for tool description
- Handles parameter validation

Troubleshooting

Common Issues

1. **Server not starting:** Check Python version and dependencies
2. **Client connection failed:** Verify server path and file permissions
3. **API errors:** Check API keys and rate limits
4. **Import errors:** Ensure virtual environment is activated

Debugging

- Check server logs for errors
- Verify API endpoints are accessible
- Test individual tools manually